

改进后的软件过程定义文档

文档版本：V2.0（融合 AI 技术）

所属项目：基于协同过滤算法的个性化电影推荐系统

撰写人：王雪琳

日期：2026年6月

说明：本文档在第一次作业《软件过程剪裁与定义》基础上更新，并吸收交付一至交付三的改进与验证结果。

1. 文档概述

1.1 编写目的

本文档用于完整描述本电影推荐 Web 项目采用的软件过程，包括各阶段活动、角色分工、交付物、质量标准及度量方式。相比第一版过程定义，本次主要增加了 AI 辅助开发相关活动，并将交付三的原型验证结论写入过程说明。

1.2 适用范围

- 项目规模：单人开发，周期约 1~2 个月
- 技术架构：B/S 结构，Django + MySQL + Bootstrap
- 核心功能：用户管理、电影浏览、评分、协同过滤推荐、AI 电影问答

1.3 过程定义依据

过程定义参考 ISO/IEC 12207 的软件生命周期划分[3]，结合瀑布模型与敏捷迭代做法。行业方面参考信通院 AI4SE 调查报告[1]和智能测试相关标准[2]，再按单人项目做剪裁。

1.4 与其他文档的关系

文档	内容
软件过程剪裁与定义（第一版）	瀑布 + Sprint + 剪裁原则
AI技术应用场景匹配报告	四个 AI 应用场景分析
软件过程改进方案设计文档	AI 融入流程与工具选型
AI技术应用验证报告	场景三测试验证数据
本文档（第二版过程定义）	整合以上内容的完整过程说明

2. 项目背景与过程目标

2.1 项目背景

本项目是一个个性化电影推荐系统，数据来源为 MovieLens 1M，推荐算法以用户协同过滤为主，物品相似度用于详情页展示。系统除传统推荐外，还增加了基于 DeepSeek API 的电影问答功能。开发中的主要难点是：推荐逻辑实现与性能、外部 API 集成、以及测试覆盖不足。

2.2 过程目标

目标	说明
功能完整	完成注册登录、浏览、评分、推荐、AI 问答等 9 类功能
过程可控	分阶段推进，每个 Sprint 结束有可演示增量
AI 辅助	在需求、编码、测试环节引入 AI，减轻重复劳动
质量可验	关键页面可手工测试，并逐步补充自动化测试

3. 过程剪裁原则

第一次作业已对标准过程做了剪裁，改进后仍然遵守以下原则：

标准活动	本项目做法	原因
正式需求评审	自行整理需求清单	单人项目，无评审组
详细设计文档	保留架构说明、ER 图、主要页面结构	避免文档过多
正式代码评审	自己检查 + AI 辅助看代码	无其他开发人员
正式验收测试	黑盒功能测试 + 部分自动化测试	无甲方，满足演示需要即可
配置管理	Git 版本管理	满足回溯需要即可

引入 AI 后新增的原则：**AI 输出一律视为草稿，必须经开发者审核后才能纳入项目。**

4. 总体过程框架

本项目采用**五阶段瀑布主线 + 四 Sprint 迭代开发**的混合过程。瀑布保证阶段顺序清晰；Sprint 用于在「迭代开发」阶段内分模块交付。改进后在需求、设计、开发、测试四个阶段各增加 AI 辅助节点。

```
flowchart TB
    subgraph 瀑布主线
        P1[1.需求分析] --> P2[2.系统设计]
        P2 --> P3[3.迭代开发]
        P3 --> P4[4.测试验证]
        P4 --> P5[5.部署交付]
    end

    subgraph AI节点
        A1[AI辅助整理需求]
        A2[AI辅助设计草案]
        A3[AI辅助编码]
        A4[AI生成测试草案]
    end

    P1 -.-> A1 -.-> P1
    P2 -.-> A2 -.-> P2
    P3 -.-> A3 -.-> P3
    P4 -.-> A4 -.-> P4
```

4.1 各阶段概要

阶段	主要活动	产出物	AI 参与
需求分析	确定功能与非功能需求	需求条目、用户故事	有
系统设计	架构、数据库、界面设计	设计说明、ER 图	有（辅助）
迭代开发	四个 Sprint 编码	可运行代码	有
测试验证	功能测试、单元测试	测试记录、测试代码	有
部署交付	本地部署、演示	可运行系统	基本无

5. 各阶段活动流程

5.1 需求分析阶段

活动： 梳理系统目标，列出 9 大功能模块，对复杂功能（如推荐）补充边界情况。

流程：

```
flowchart LR
    S1[列出功能点] --> S2[AI整理用户故事]
    S2 --> S3[本人核对]
    S3 --> S4[确定需求条目]
```

AI 应用（场景一）： 将推荐功能描述发给 DeepSeek，请其补充冷启动、推荐为空等情况，再对照 RecommendMovieView 确认。Sprint 3 开发时曾遗漏新用户无评分场景，后续需求阶段需提前考虑。

交付物： 功能需求列表、关键用户故事。

质量标准： 主流程清晰，复杂模块至少考虑 2~3 种边界情况。

示例（推荐模块）： 向 AI 描述「基于用户协同过滤，K=20，N=10」，可得到冷启动、相似用户不足、推荐列表为空等条目，再决定哪些写入本次 Sprint 的实现范围。

5.2 系统设计阶段

活动： 确定 B/S 三层结构、Django 应用划分、数据库表关系、主要页面跳转。

流程： 开发者确定方案 → AI 生成 ER 图或模块说明初稿 → 对照 models.py 修改定稿。

AI 应用： 辅助出图和文字说明，不替代架构决策。

交付物： 系统架构说明、数据库 ER 图、主要页面结构。

质量标准： 表结构与代码中 Model 一致，模块划分支持四个 Sprint。

5.3 迭代开发阶段

开发阶段划分为四个 Sprint，每个 Sprint 内可按「AI 辅助编码 → 本人调试 → 小范围测试」推进。

Sprint	模块	主要功能	AI 使用情况
Sprint 1	用户模块	注册、登录、登出	常规编码为主
Sprint 2	电影展示	首页、分类、搜索、热门	常规编码为主
Sprint 3	评分与推荐	评分、历史、UserCF 推荐	需求与代码优化拟用 AI
Sprint 4	AI 问答	DeepSeek 流式问答	已用 Cursor 完成初版

```
flowchart LR
    C1[确定Sprint任务] --> C2[AI生成或修改代码]
    C2 --> C3[本地运行调试]
    C3 --> C4[Sprint演示]
```

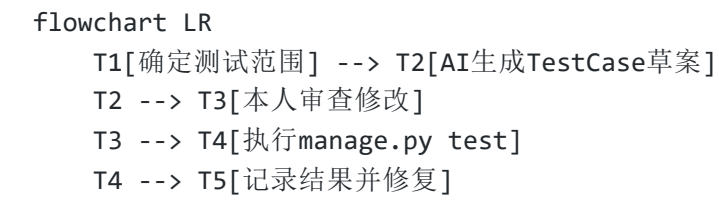
AI 应用（场景二、场景四）：

- 场景四：Cursor 生成 AIChatView 、 ai_chat.html 初稿，本人修改 API 与样式。
- 场景二： RecommendMovieView 存在循环查库问题，计划结合 tests.py 中 UserCFRec 思路请 AI 提优化建议，由本人实现。

5.4 测试验证阶段

活动： 对每个 Sprint 成果做功能测试；逐步补充 Django 自动化测试。

流程：



AI 应用（场景三）及验证结论：

交付三已对场景三做验证：用 Cursor 生成测试草案，修改后写入 `movie/tests_views.py`，本地执行 `python manage.py test movie.tests_views` 得到 7 passed。耗时为本人估算，非严格对比实验；生成代码需删外网用例、改表单字段后才能用。

交付物： 测试用例列表、测试代码、测试执行记录。
质量标准： 核心页面可访问；关键逻辑有测试覆盖；AI 生成测试须审查断言。

验证记录摘要： Prompt 中需写明「不调用真实 DeepSeek API」；删除 AI 初稿中会访问外网的用例；登录测试字段须与 `LoginForm` 一致。执行命令为 `python manage.py test movie.tests_views`，通过后再并入常规测试流程。

持续测试： 开发过程中边写边测，每个 Sprint 结束至少完成该模块的手工走查；有余力时用 AI 补自动化测试，不追求一次写完全部用例。

5.5 部署交付阶段

活动： 配置 MySQL、导入数据、本地启动 Django、准备演示。

AI 应用： 仅在排查报错时偶发使用，不作强制环节。
交付物： 可运行系统、过程与实验相关文档。
质量标准： 主要功能可演示，无明显阻塞性缺陷。

6. 功能模块划分

系统共 9 个功能模块，与 Sprint 对应关系如下：

编号	模块	所属 Sprint
1	用户注册登录	Sprint 1
2	电影首页展示	Sprint 2
3	分类浏览	Sprint 2
4	搜索	Sprint 2

编号	模块	所属 Sprint
5	电影评分	Sprint 3
6	评分历史	Sprint 3
7	热门电影	Sprint 2
8	个性化推荐	Sprint 3
9	AI 电影问答	Sprint 4

7. 角色与职责

本项目为单人开发，角色由本人兼任，AI 工具作为虚拟辅助角色。

角色	职责
开发者	计划、设计决策、编码、测试、部署、验收
AI 需求辅助	整理用户故事，提示边界场景
AI 编码辅助	生成代码初稿、提示性能问题
AI 测试辅助	生成 TestCase 草案

人机分工：开发者定义目标和验收标准；AI 生成草稿；开发者审核、修改并集成。不使用自训练模型，故不设「AI 训练师」岗位。

8. 工具链

用途	工具
需求整理	DeepSeek 网页版
编码	Cursor
AI 问答运行时	DeepSeek API
测试	Django TestCase + Cursor
版本管理	Git
数据库	MySQL
Web 框架	Django

使用约定： Cursor 生成代码后必须逐段检查；DeepSeek 对话不粘贴 API Key 和数据库密码；测试与算法实验代码分文件存放（如 tests.py 与 tests_views.py ），避免互相干扰。

9. 交付物清单与质量标准

9.1 各阶段交付物

阶段	交付物	质量标准
需求分析	需求说明	功能完整、边界情况有记录
系统设计	架构与 ER 图	与实现一致
迭代开发	源代码	可运行，符合模块划分
测试验证	测试代码与记录	核心流程可测，测试能执行
部署交付	部署系统	本地可访问

9.2 AI 相关交付物验收

AI 输出	验收方式
需求条目	逐条核对，删除不适用项
代码	本地运行，对照项目结构
测试草案	执行测试命令，检查断言
设计草案	与 Model、路由对照

10. 过程度量指标

结合第一次作业与交付三验证，采用以下指标跟踪过程效果。数值以「记录实际情况」为主，不作过高承诺。

编号	指标	说明	当前情况
M1	Sprint 完成数	4 个 Sprint 是否按期有可演示成果	4 个已完成
M2	功能模块完成率	9 个模块实现比例	9/9 已实现
M3	自动化测试用例数	tests_views.py 等 TestCase 数量	7 个（验证后）
M4	自动化测试执行情况	tests_views.py 能否跑通	7 个用例通过

编号	指标	说明	当前情况
M5	推荐页响应	访问推荐页主观感受	偏慢，待优化
M6	AI 测试草案可用性	是否需大量修改才能跑通	需删外网用例、改字段

11. 风险与应对

风险	应对
API Key、密码泄露	尽量用环境变量，提交前检查
用户数据发给 AI	不使用真实数据，测试用虚构账号
AI 代码错误	必须本地验证，核心算法自己掌握
外部 API 不可用	页面提示错误，不影响其他功能
过度依赖 AI	协同过滤原理能自己说明
测试用例质量差	审查断言，补边界场景

12. 四个 AI 应用场景在本过程中的位置

场景	阶段	状态
场景一：推荐需求整理	需求分析	已纳入流程，Sprint 3 曾部分遗漏，后续补做
场景二：推荐代码优化	编码实现	已纳入流程，性能优化进行中
场景三：View 测试编写	测试验证	已纳入流程，交付三已验证
场景四：AI 问答开发	编码实现	已完成，Sprint 4 交付

四个场景覆盖需求、编码、测试三个阶段，设计阶段仅作辅助，运维阶段暂不强制使用 AI。

13. 过程改进前后对比

方面	改进前（第一版）	改进后（本版）
AI 使用	临时求助	写入各阶段流程
测试	以手工点击为主	增加 AI 辅助生成 TestCase

方面	改进前（第一版）	改进后（本版）
需求	边做边补	推荐等复杂功能先经 AI 整理
文档	过程定义	过程定义 + 场景匹配 + 验证记录

14. 小结

改进后的软件过程仍保持「瀑布 + 四 Sprint + 过程剪裁」的基本结构，在需求、设计、开发、测试阶段嵌入了 AI 辅助节点，并通过交付三对测试场景做了实际验证。实践表明，AI 适合承担草稿生成和常见问题提示，审核和决策仍由开发者完成。

后续计划：继续用 AI 辅助优化推荐页性能，并按每个 Sprint 补充 View 层测试；API Key 改为环境变量配置。本过程定义将随项目进展做小版本更新。

从个人体会看，AI 并没有减少总工作量，而是把查文档、写初稿的时间挪到了审查和修改上。对单人项目来说，这样仍然值得，因为更容易发现遗漏的边界情况和测试场景。过程文档写到 V2.0 为止，若推荐模块完成性能优化，会在度量指标中补充响应时间数据。

编码阶段用到的 Cursor、Copilot 类工具，以及 AI 问答用的 DeepSeek API，官方文档里对使用方式有说明[4][5]，和本项目实际选型一致。过程框架仍以 ISO/IEC 12207 的阶段划分为基础[3]，在上面做剪裁和 AI 节点嵌入，而不是另起一套流程。各阶段交付物以能演示、能跑通为底线，不额外堆很多无用文档。

参考文献

[1] 中国信息通信研究院. AI4SE行业现状调查报告（2024年度）[R]. 2024.

[2] 中国信息通信研究院. 智能化软件工程技术和应用要求 第3部分：智能测试能力（AIIA/PG 0138-2024）[S]. 2024.

[3] ISO/IEC. ISO/IEC 12207:2017 系统与软件工程——软件生命周期过程[S]. 2017.

[4] GitHub, Inc. GitHub Copilot 文档[EB/OL]. <https://docs.github.com/zh/copilot>, 2024.

[5] DeepSeek. DeepSeek API 文档[EB/OL]. <https://api-docs.deepseek.com>, 2024.